ESR — Trabalho Prático 2

Plano Mestre de Desenvolvimento: Serviço OTT para Distribuição de Multimédia

Universidade do Minho | Engenharia de Serviços em Rede | 2026/2027

Objetivo: Projetar, implementar, testar e justificar cientificamente uma rede *Over-the-Top* (OTT) para distribuição eficiente, escalável, resiliente e adaptativa de conteúdos multimédia.

Referências fundamentais:

- Andrew S. Tanenbaum — *Computer Networks*.
- James F. Kurose e Keith W. Ross — *Computer Networking: A Top-Down Approach*.
- Documentação oficial do CORE (*Common Open Research Emulator*).
- Enunciado oficial do Trabalho Prático 2.

PARTE I — Compreender o problema antes de desenvolver

Concordo com a abordagem que propões: **não vamos começar a implementar sem compreender a arquitetura completa, as decisões técnicas e o funcionamento do sistema.**

Também não vamos procurar apenas uma solução que funcione numa topologia específica e durante uma demonstração.

Vamos procurar desenvolver um sistema fundamentado nos princípios de redes de computadores, capaz de se adaptar a diferentes topologias, números de clientes, localizações e condições de funcionamento.

Entretanto, precisamos estabelecer uma distinção fundamental:

> O nosso objetivo não é criar um sistema que nunca falhe. É desenvolver um sistema que detete alterações e falhas, se adapte às condições disponíveis, utilize eficientemente os recursos e mantenha a melhor qualidade de serviço possível.

Nenhuma arquitetura consegue garantir qualidade ilimitada para qualquer quantidade de clientes: existem limites físicos de largura de banda, processamento, memória e conectividade.

O que podemos fazer é projetar uma arquitetura que minimize esses limites e degrade o serviço de forma controlada quando necessário.

1. Qual é exatamente o problema?

Imaginemos uma rede com:

- 2 servidores multimédia;

- 20 clientes;
- 8 nós intermédios;
- vários routers;
- várias ligações com diferentes capacidades e atrasos.

Os clientes pretendem assistir a diferentes vídeos transmitidos pelos servidores.

1.1. Primeira abordagem: comunicação direta

Numa arquitetura tradicional de *unicast*, cada cliente estabelece uma comunicação independente com o servidor.

Se 20 clientes assistirem ao mesmo vídeo, poderão existir 20 transmissões idênticas a partir do servidor.

Consideremos um vídeo com taxa de transmissão de 4 Mbps.

Clientes	Tráfego de saída do servidor
1	4 Mbps
5	20 Mbps
10	40 Mbps
20	80 Mbps
100	400 Mbps
1 000	4 Gbps

Estes valores assumem um fluxo constante, sem contabilizar overhead protocolar e retransmissões.

Qual é o problema?

O servidor transmite repetidamente o mesmo conteúdo, desperdiçando recursos computacionais e largura de banda.

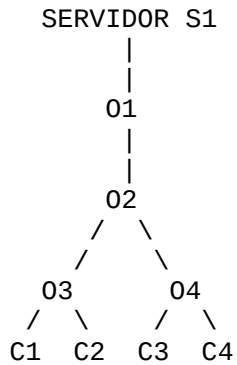
Além disso, algumas ligações da infraestrutura poderão transportar repetidamente os mesmos dados.

Este é precisamente um dos problemas abordados no enunciado.

1.2. Segunda abordagem: distribuição através de uma rede overlay

Em vez de cada cliente receber diretamente os dados do servidor, vamos criar uma rede lógica de distribuição.

Por exemplo:



Neste exemplo:

- S1 produz o conteúdo.
- O1 recebe o fluxo.
- O2 encaminha-o para dois ramos.
- O3 e O4 replicam os dados apenas quando necessário.
- C1, C2, C3 e C4 recebem o vídeo.

A mesma ligação lógica entre O1 e O2 pode transportar apenas uma cópia do fluxo, independentemente dos quatro clientes representados.

A replicação acontece progressivamente, nos pontos onde os caminhos se dividem.

Fundamento: multicast aplicacional, distribuição em árvore e redução de tráfego redundante.

Esta abordagem está relacionada com os princípios de encaminhamento e distribuição multicast abordados na literatura de redes.

É importante compreender que estamos a criar multicast **na camada aplicacional**, não a ativar necessariamente IP multicast nos routers físicos.

---

2. A nossa visão da solução completa

Proponho que desenvolvamos uma arquitetura de **overlay multicast aplicacional, dinâmica e consciente das condições da rede**.

Isto significa que os nós não serão apenas retransmissores de pacotes.

Terão responsabilidades de descoberta, controlo, encaminhamento, monitorização e recuperação.

A solução terá seis componentes fundamentais.

Componente	Responsabilidade	Conceito fundamental
Descoberta	Encontrar nós e permitir entrada na rede	Bootstrap e descoberta de serviços
Gestão do overlay	Construir e manter ligações entre vizinhos	Grafos e topologias lógicas
Encaminhamento	Selecionar caminhos para os fluxos	Algoritmos de routing
Distribuição	Replicar os dados apenas onde necessário	Multicast aplicacional
Monitorização	Medir desempenho e detetar alterações	QoS e gestão de redes
Recuperação	Reconstruir rotas após falhas	Resiliência e convergência

Esses componentes deverão funcionar em conjunto.

2.1. Funcionamento pretendido

Imaginemos a seguinte sequência:

Situação A — A rede arranca

Os servidores iniciam a execução, seguidos dos nós intermédios.

Os nós descobrem os seus vizinhos e estabelecem o overlay.

Situação B — Um cliente entra

O cliente inicia a aplicação, descobre a rede e solicita um determinado vídeo.

O sistema encontra uma rota adequada.

Situação C — Entram outros clientes

Novos clientes podem utilizar partes das rotas existentes, evitando transmissões desnecessárias.

Situação D — Uma ligação fica congestionada

O sistema deteta uma alteração nas condições da rede.

Se existir um caminho alternativo viável, pode atualizar a rota.

Situação E — Um nó intermédio falha

Os vizinhos detetam a falha.

O sistema tenta restabelecer o caminho através de outros nós.

Situação F — Um cliente abandona o serviço

O sistema atualiza o estado de subscrição e elimina encaminhamentos desnecessários.

Situação G — Existem vários servidores e vários conteúdos

Cada cliente pode selecionar o conteúdo pretendido, e o sistema mantém o encaminhamento dos diferentes fluxos sem os confundir.

Essa será a visão global que orientará as decisões de implementação.

---

3. Requisitos obrigatórios do enunciado

Há um ponto particularmente importante: o trabalho exige explicitamente **pelo menos dois servidores, cada um disponibilizando no mínimo dois fluxos de vídeo distintos**.

Também exige que sejam utilizadas métricas da rede na construção das rotas.

Logo, não podemos terminar com uma solução funcional para um único servidor e um único fluxo, mesmo que esse seja o cenário inicial de desenvolvimento.

A tabela seguinte distingue os requisitos do enunciado das funcionalidades adicionais que proponho.

Requisito	Obrigatório	Nossa abordagem
Emulação com CORE	Sim, ambiente principal	Utilizar o CORE
Nós overlay executados como aplicações	Sim	Aplicação independente
Entrada e saída dinâmica de nós	Sim	Registo e remoção
Descoberta e manutenção de vizinhos	Sim	Bootstrap e monitorização
Tabelas de encaminhamento	Sim	Tabelas por fluxo
Rotas alternativas	Sim	Caminhos alternativos
Vários clientes simultâneos	Sim	Distribuição multicast

Requisito	Obrigatório	Nossa abordagem
Atualização dinâmica das rotas	Sim	Monitorização e recálculo
Encaminhamento de dados pelas tabelas	Sim	Plano de dados separado
Dois servidores	Sim	S1 e S2
Dois vídeos por servidor	Sim	Quatro fluxos inicialmente
Métricas de rede para seleção de caminhos	Sim	Custo baseado em métricas
Streaming real	Sim	Transmissão e reprodução
Testes com diferentes topologias	Sim, como parte da avaliação	Três cenários-base e testes adicionais
Persistência das configurações	Melhoria nossa	Configuração declarativa
Registos estruturados	Melhoria nossa	Logs por nó
Métricas de experiência do utilizador	Melhoria nossa	Medições no cliente
Recuperação automática avançada	Extensão do requisito de atualização dinâmica	Recuperação e comparação experimental

Atenção: a avaliação inclui 10% para topologia, 25% para gestão dos fluxos, 15% para protocolo de controlo, 10% para streaming, 30% para solução integrada e duas avaliações intermédias de 5% cada.

Portanto, devemos dedicar especial atenção à **gestão de fluxos** e à **integração de todos os componentes**.

Fonte: enunciado oficial, páginas 2 a 5. ESR-TP2-26-27 ESR-TP2-26-27

PARTE II — Decisões de arquitetura que precisamos estudar

Antes de programarmos, precisamos responder a algumas perguntas.

Não basta escolher uma tecnologia porque é popular ou porque facilita a implementação. Cada escolha terá de ser justificada.

4. UDP ou TCP?

Esta será uma das primeiras decisões.

A escolha influencia a latência, a recuperação de perdas, a gestão da congestão e a própria arquitetura do sistema.

Característica	TCP	UDP
Orientado a conexão	Sim	Não
Entrega ordenada	Sim	Não garante
Retransmissão automática	Sim	Não
Controlo de congestão integrado	Sim	Não

Característica	TCP	UDP
Preservação dos limites das mensagens	Não	Sim, por datagrama
Adequação a streaming	Possível	Possível
Responsabilidade da aplicação	Menor	Maior

A minha proposta inicial

Considerarmos duas alternativas:

Alternativa A: TCP para controlo e UDP para streaming.

O TCP transportaria mensagens de registo, informações sobre vizinhos e outras mensagens de controlo que beneficiam de entrega fiável.

O UDP transportaria os pacotes multimédia, permitindo à aplicação controlar o comportamento temporal do streaming.

Alternativa B: UDP para controlo e streaming.

Neste caso, precisaríamos de implementar mecanismos de fiabilidade para algumas mensagens de controlo.

Inicialmente, considero a alternativa A mais adequada ao nosso projeto.

Contudo, não vamos adotá-la definitivamente sem estudar as implicações.

UDP não significa automaticamente menor latência, e TCP não significa automaticamente pior streaming.

Por exemplo, aplicações modernas utilizam streaming sobre HTTP/TCP e também protocolos baseados em UDP, como QUIC.

O que importa é compreender os requisitos de cada comunicação.

5. Como construir o overlay?

Temos três abordagens principais para estudar.

Abordagem	Vantagem	Limitação
Configuração manual	Simples e previsível	Pouca adaptação
Controlador central	Visão global e gestão simplificada	Dependência do controlador
Distribuída	Maior autonomia dos nós	Maior complexidade

O enunciado propõe explicitamente estratégias de configuração manual e utilização de um *bootstrapper.*

Arquitetura inicial recomendada

Proponho começar com um **bootstrapper centralizado para descoberta**, mantendo o encaminhamento e a distribuição dos fluxos nos próprios nós overlay.

Esta separação é importante.

O bootstrapper não deve transportar obrigatoriamente todos os vídeos.

A sua função principal será permitir que os nós descubram a rede.

Depois, os nós executam as tarefas de distribuição.

Posteriormente, podemos estudar até que ponto o sistema deve depender do bootstrapper para atualizar rotas e gerir falhas.

A arquitetura precisa de continuar a funcionar perante falhas de nós intermédios, e devemos testar explicitamente o comportamento quando o próprio bootstrapper fica indisponível.

6. Como escolher as melhores rotas?

Este será um dos elementos cientificamente mais importantes do trabalho.

Não queremos selecionar caminhos apenas por conveniência.

Vamos estudar:

Número de saltos

Uma métrica simples.

$$ C(P)=\sum_{e\in P}1 $$

Atraso

Considera o atraso observado nas ligações ou nos percursos.

$$ C(P)=\sum_{e\in P}d_e $$

Capacidade e largura de banda disponível

Permite avaliar se determinado percurso suporta um fluxo.

Para um percurso P, a capacidade disponível fica limitada pelo seu segmento mais restritivo:

$$ B(P)=\min_{e\in P} B_e $$

Aqui, B_e representa uma estimativa da largura de banda disponível no segmento e.

Métrica composta

Podemos ainda estudar uma função que considere vários fatores:

$$ C(P)=\sum_{e\in P} \left(\alpha D_e+ \beta L_e+ \gamma U_e \right) $$

Onde:

- D_e: atraso normalizado;
- L_e: perda de pacotes normalizada;
- U_e: utilização da ligação;
- α,β,γ: pesos das métricas.

Esta expressão é uma proposta de modelação nossa, não uma fórmula obrigatória do enunciado.

Precisaremos de investigar como medir cada variável, normalizar os valores e evitar alterações excessivas de rota.

Também precisamos de distinguir duas otimizações diferentes:

1. Encontrar o melhor caminho entre um servidor e um cliente.

2. Encontrar uma boa estrutura de distribuição para vários clientes, evitando duplicação de tráfego.

O caminho individualmente mais rápido nem sempre conduz à árvore de distribuição mais eficiente.

Esta distinção será central na fundamentação com Kurose e Tanenbaum.

PARTE III — Roteiro de execução por checkpoints

Vamos dividir o trabalho em **oito checkpoints técnicos**.

Esta é a nossa organização interna. O enunciado prevê avaliações intermédias, mas não especifica, nas páginas fornecidas, que estas oito etapas correspondam às datas oficiais dessas avaliações.

A cada checkpoint teremos quatro resultados:

1. Conhecimento teórico consolidado.
2. Implementação funcional.
3. Testes reproduzíveis.
4. Documentação para o relatório e para a apresentação.

Regra do projeto: nenhum checkpoint será considerado concluído apenas porque o programa executou sem apresentar erros.

Precisamos de demonstrar que o comportamento observado corresponde ao comportamento esperado.

✅ CHECKPOINT 0 — Fundamentos, arquitetura e preparação

Objetivo: compreendermos a problemática e definirmos uma arquitetura justificável antes de implementar.

A. Conhecimento teórico

- Compreender os modelos OSI e TCP/IP.
- Rever comunicação cliente-servidor.
- Compreender sockets TCP e UDP.
- Estudar endereçamento IPv4 e sub-redes.
- Compreender encaminhamento IP.
- Distinguir plano de controlo e plano de dados.
- Estudar unicast, broadcast e multicast.
- Compreender o conceito de overlay.
- Compreender a relação overlay/underlay.

- Estudar o funcionamento básico de CDNs.
- Compreender as diferenças entre QoS e QoE.
- Rever conceitos de largura de banda, débito, latência, jitter e perda de pacotes.
- Compreender os requisitos de streaming em tempo real.

B. Compreensão do problema

- Identificar todos os requisitos obrigatórios.
- Identificar os comportamentos desejáveis adicionais.
- Definir os tipos de falhas que pretendemos suportar.
- Identificar as limitações de recursos.
- Estabelecer objetivos de desempenho mensuráveis.
- Definir o âmbito da solução.
- Identificar os riscos técnicos.

C. Decisões arquiteturais

- Escolher a linguagem de programação.
- Justificar a linguagem escolhida.
- Definir o papel dos servidores.
- Definir o papel dos nós overlay.
- Definir o papel dos clientes.
- Decidir como os nós descobrem a rede.
- Definir uma estratégia inicial de construção do overlay.
- Escolher os protocolos de transporte.
- Definir a identidade de cada nó.
- Definir a identidade dos fluxos.
- Especificar a separação entre controlo e dados.
- Elaborar o primeiro diagrama de arquitetura.

D. Preparação do ambiente

- Verificar a versão do sistema operativo.
- Instalar e validar o CORE.
- Identificar a versão instalada do CORE.
- Instalar as dependências de desenvolvimento.
- Confirmar a comunicação entre nós emulados.
- Criar o repositório Git.
- Organizar as pastas do código.
- Organizar o projeto LaTeX.
- Definir convenções para nomes, código, logs e configurações.
- Criar a primeira topologia de testes.
- Guardar essa topologia para reutilização.

Entregáveis do checkpoint 0

Entregável	Descrição
Documento de arquitetura	Visão global e decisões iniciais

Entregável	Descrição
Diagrama do sistema	Componentes e comunicações
Ambiente CORE	Emulador funcional
Primeira topologia	Topologia guardada
Repositório	Estrutura organizada
Matriz de requisitos	Requisitos e testes correspondentes

Critério de aprovação: todos os três elementos devem conseguir explicar a arquitetura proposta e demonstrar a comunicação IP entre dispositivos da topologia.

✅ CHECKPOINT 1 — Construção e configuração persistente da infraestrutura

Objetivo: criar uma infraestrutura emulada que possa ser iniciada, terminada e reconstruída sem configuração manual repetitiva.

Este checkpoint responde diretamente a uma das tuas maiores preocupações.

O problema da configuração manual

Imagina que criamos uma rede com 15 dispositivos.

Configuramos os endereços, as rotas IP, as ligações e os serviços.

Depois desligamos o CORE.

Se tivermos de repetir tudo manualmente na sessão seguinte, isso representa uma perda de tempo e introduz erros.

A solução será utilizar **infraestrutura e configurações reprodutíveis**.

A. Infraestrutura

- Criar a topologia-base no CORE.
- Identificar os dispositivos.
- Definir os segmentos de rede.
- Definir o plano de endereçamento IPv4.
- Configurar as interfaces.
- Configurar as rotas IP necessárias.
- Validar a conectividade entre os nós terminais.
- Confirmar os caminhos reais utilizados pelo underlay.
- Definir a capacidade das ligações.
- Definir atrasos e perdas nos enlaces de teste.

B. Persistência

- Guardar a topologia no formato suportado pela versão do CORE.
- Guardar as configurações das interfaces.

- Guardar as configurações de encaminhamento.
- Definir quais os serviços que arrancam automaticamente.
- Criar ficheiros de configuração para os nós overlay.
- Guardar as configurações de servidores e clientes.
- Centralizar os parâmetros comuns.
- Separar as configurações dos diferentes cenários.
- Criar um mecanismo de arranque automatizado.
- Criar um mecanismo de limpeza e encerramento.
- Confirmar que os logs são exportados para armazenamento persistente.

C. Testes de reprodução

- Iniciar a topologia.
- Validar todos os endereços.
- Validar conectividade.
- Terminar a sessão.
- Reiniciar o ambiente.
- Reabrir a topologia.
- Confirmar que as configurações foram restauradas.
- Executar os testes novamente.
- Confirmar que os resultados continuam coerentes.

Como vamos organizar as configurações?

Proponho uma estrutura semelhante a esta:

```
infraestrutura/
│
├── topologias/
│   ├── cenario_01/
│   ├── cenario_02/
│   ├── cenario_03/
│   └── cenario_escalabilidade/
│
├── configuracoes/
│   ├── servidores/
│   ├── clientes/
│   ├── overlay/
│   └── underlay/
│
├── scripts/
│   ├── iniciar.sh
│   ├── parar.sh
│   ├── validar.sh
│   └── limpar.sh
│
└── README.md
```

Não vamos depender apenas de alterações realizadas pela interface gráfica do CORE.

A versão exata do CORE determinará quais os formatos de topologia, serviços e mecanismos de automatização que utilizaremos.

As configurações serão guardadas fora dos sistemas temporários dos nós emulados, pois não devemos pressupor que alterações feitas durante uma sessão persistem automaticamente.

Critério de aprovação: conseguir reconstruir a infraestrutura desde o início e executar um teste de conectividade sem reconfigurar manualmente os dispositivos.

✅ CHECKPOINT 2 — Descoberta e construção dinâmica do overlay

Objetivo: fazer com que diferentes nós consigam entrar numa rede lógica e manter relações com os seus vizinhos.

A. Fundamentos

- Estudar grafos e representação de topologias.
- Compreender vizinhança lógica e física.
- Estudar bootstrap e descoberta de serviços.
- Compreender sockets de escuta e ligação.
- Estudar concorrência e comunicações full-duplex.
- Compreender mecanismos de deteção de falhas.

B. Protocolo de controlo

- Definir o formato das mensagens.
- Definir campos obrigatórios.
- Criar mensagens `HELLO`.
- Criar mensagens de registo.
- Criar respostas contendo os vizinhos.
- Implementar confirmação de ligações.
- Implementar mensagens de manutenção.
- Implementar mensagens de saída.
- Definir identificadores únicos de mensagens.
- Definir tratamento de mensagens inválidas.
- Definir mecanismos para evitar processamento duplicado.

C. Implementação

- Implementar o bootstrapper.
- Implementar o processo de registo.
- Criar a tabela de vizinhos.
- Implementar comunicação entre vizinhos.
- Permitir entrada dinâmica de novos nós.
- Permitir saída voluntária.
- Detetar falhas inesperadas.
- Atualizar a tabela de vizinhos.
- Registar eventos em logs.

D. Testes

- Overlay com dois nós.
- Overlay com quatro nós.
- Overlay com seis nós.
- Entrada de um nó durante a execução.
- Saída voluntária de um nó.
- Desligamento inesperado de um nó.
- Tentativa de registo duplicado.
- Bootstrapper temporariamente indisponível.
- Recuperação de conectividade após falha.

Critério de aprovação: criar e modificar dinamicamente a topologia lógica sem precisar de reconstruir manualmente toda a infraestrutura.

✅ CHECKPOINT 3 — Descoberta de fluxos e construção de rotas

Objetivo: permitir que os nós encontrem caminhos adequados para os conteúdos disponíveis.

A. Fundamentos

- Rever algoritmos de encaminhamento.
- Estudar flooding controlado.
- Compreender prevenção de ciclos.
- Estudar grafos ponderados.
- Rever o algoritmo de Dijkstra.
- Compreender distribuição de custos.
- Estudar convergência de rotas.
- Estudar o problema de caminhos redundantes.
- Compreender árvores de distribuição multicast.

B. Gestão de fluxos

- Criar identificadores únicos para servidores.
- Criar identificadores únicos para fluxos.
- Implementar o catálogo de conteúdos.
- Suportar no mínimo dois servidores.
- Suportar no mínimo dois vídeos por servidor.
- Implementar anúncios de disponibilidade.
- Implementar subscrição de clientes.
- Implementar cancelamento de subscrição.

C. Encaminhamento

- Definir a estrutura da tabela de rotas.
- Guardar origem, destinos e estado.

- Associar cada rota a um fluxo específico.
- Implementar uma métrica inicial.
- Calcular rotas.
- Evitar ciclos.
- Permitir rotas alternativas.
- Ativar percursos apenas quando necessários.
- Desativar percursos sem clientes.
- Atualizar rotas quando a rede muda.

D. Testes

- Cliente com caminho único.
- Cliente com caminhos alternativos.
- Vários clientes para o mesmo fluxo.
- Clientes com fluxos diferentes.
- Dois servidores ativos.
- Alteração da topologia overlay.
- Alteração da métrica de uma ligação.
- Verificação da ausência de ciclos.
- Verificação de encaminhamento não redundante.

Critério de aprovação: cada cliente deve conseguir encontrar uma rota válida para o fluxo solicitado, e os nós devem manter corretamente as respetivas tabelas de encaminhamento.

✅ CHECKPOINT 4 — Distribuição dos dados e streaming

Objetivo: transmitir multimédia real através do overlay.

A. Fundamentos

- Compreender codificação e formatos de vídeo.
- Estudar segmentação e encapsulamento.
- Compreender números de sequência.
- Estudar controlo de ritmo de transmissão.
- Compreender buffering.
- Estudar jitter.
- Compreender perda e reordenação de pacotes.
- Estudar técnicas básicas de recuperação.
- Identificar diferenças entre streaming ao vivo e conteúdo pré-gravado.

B. Protocolo de dados

- Definir o formato dos pacotes multimédia.
- Adicionar identificação do fluxo.
- Adicionar números de sequência.

- Definir informação temporal.
- Estabelecer um tamanho máximo apropriado para os pacotes.
- Implementar segmentação quando necessária.
- Implementar reagrupamento no destino.
- Implementar identificação de pacotes perdidos.
- Definir comportamento perante atraso excessivo.

C. Distribuição

- Implementar emissão no servidor.
- Implementar receção nos nós overlay.
- Consultar a tabela de encaminhamento.
- Encaminhar dados para os destinos corretos.
- Replicar pacotes apenas nos pontos necessários.
- Impedir encaminhamento para clientes não subscritos.
- Suportar vários fluxos simultâneos.
- Implementar reprodução no cliente.

D. Testes

- Um servidor e um cliente.
- Um servidor e múltiplos clientes.
- Dois servidores com fluxos simultâneos.
- Clientes a assistir a conteúdos diferentes.
- Clientes a assistir ao mesmo conteúdo.
- Transmissão com atraso artificial.
- Transmissão com perda artificial.
- Transmissão com capacidade limitada.
- Análise da fluidez da reprodução.

Critério de aprovação: todos os clientes inscritos devem conseguir receber e reproduzir o conteúdo solicitado, respeitando os limites de capacidade da rede.

✅ CHECKPOINT 5 — Monitorização, adaptação e recuperação

Objetivo: transformar um overlay funcional num overlay adaptativo.

Este será um dos checkpoints mais interessantes do ponto de vista de engenharia.

A. Monitorização

- Medir RTT entre nós.
- Estimar perdas de pacotes.
- Monitorizar a disponibilidade dos vizinhos.
- Medir o débito observado.

- Estimar utilização ou disponibilidade de capacidade quando possível.
- Guardar histórico de medições.
- Identificar degradação de desempenho.
- Identificar falhas de comunicação.

B. Adaptação

- Definir uma função de custo.
- Avaliar percursos alternativos.
- Definir condições para alterar uma rota.
- Evitar oscilações frequentes de encaminhamento.
- Evitar loops durante atualizações.
- Atualizar os próximos saltos.
- Preservar as subscrições ativas.
- Minimizar interrupções durante a transição.

C. Resiliência

- Recuperar de falha de nó intermédio.
- Recuperar de falha de uma ligação.
- Tratar perda de conectividade ao bootstrapper.
- Detetar saída inesperada de clientes.
- Remover rotas obsoletas.
- Permitir reentrada de nós.
- Registar os tempos de recuperação.

D. Testes

- Introduzir congestionamento.
- Aumentar o atraso de uma ligação.
- Reduzir a capacidade de um enlace.
- Introduzir perdas.
- Desligar um nó intermédio.
- Restaurar o nó.
- Desligar uma ligação.
- Restaurar a ligação.
- Medir os tempos de deteção e recuperação.
- Avaliar a continuidade do streaming.

Critério de aprovação: o sistema deve identificar mudanças relevantes e atualizar as rotas quando existirem alternativas viáveis, sem intervenção manual.

✅ CHECKPOINT 6 — Avaliação experimental e escalabilidade

Objetivo: demonstrar, através de medições, se a solução desenvolvida realmente melhora a distribuição dos conteúdos.

Não será suficiente afirmar que a rede é eficiente.

Precisaremos de números.

A. Cenários previstos

O enunciado apresenta três cenários de teste de referência, que vamos utilizar como ponto de partida.

Cenário	Composição	Objetivo
1	4 nós: 1 servidor, 2 clientes, 1 intermédio	Validação inicial
2	6 nós: 1 servidor, 2 clientes, 3 intermédios	Rotas e encaminhamento
3	Overlay complexo	Integração e comportamento dinâmico
4	2 servidores e vários fluxos	Verificação dos requisitos completos
5	Número crescente de clientes	Escalabilidade
6	Falhas de nós e ligações	Resiliência
7	Congestionamento e atrasos	Adaptação
8	Clientes distribuídos em diferentes regiões da topologia	Independência da localização

Os três primeiros têm origem no enunciado; os restantes são extensões propostas.

B. Métricas experimentais

- Débito de dados recebido.
- Atraso fim a fim.
- Jitter.
- Percentagem de perda de pacotes.
- Tempo de entrada de um cliente.
- Tempo de descoberta de um fluxo.
- Tempo de ativação de rota.
- Tempo de recuperação após falha.
- Quantidade de tráfego duplicado.
- Tráfego total gerado pelos servidores.
- Consumo de largura de banda nas ligações.
- Utilização de CPU e memória, quando relevante.
- Interrupções e duração do buffering no cliente.

Para vídeo pré-gravado, podemos também avaliar a continuidade da reprodução, distinguindo a qualidade da rede da qualidade efetivamente percecionada pelo utilizador.

C. Comparações fundamentais

Vamos comparar pelo menos:

Experiência 1 — Unicast direto versus overlay

Verificar quanto tráfego é gerado em cada abordagem.

Experiência 2 — Rotas por número de saltos versus rotas por métricas

Avaliar se a seleção de caminhos apresenta melhorias.

Experiência 3 — Rede estática versus rede com adaptação

Verificar o comportamento quando ocorrem falhas ou degradação das ligações.

Experiência 4 — Diferentes quantidades de clientes

Observar como crescem o tráfego, o consumo de recursos e os atrasos.

Experiência 5 — Diferentes distribuições geográficas lógicas dos clientes

Avaliar o impacto de posicionar os clientes em regiões diferentes da infraestrutura emulada.

Uma regra essencial

Todos os resultados deverão ser reproduzíveis.

- Guardar a configuração de cada experiência.
- Registar a versão do software.
- Definir a duração dos ensaios.
- Repetir os testes.
- Identificar possíveis fatores de variação.
- Apresentar médias e dispersão quando apropriado.
- Gerar gráficos comparativos.
- Interpretar os resultados.
- Discutir limitações.

Critério de aprovação: ter evidências quantitativas de eficiência, escalabilidade, comportamento perante falhas e qualidade de streaming.

✅ CHECKPOINT 7 — Integração final, relatório e defesa

Objetivo: apresentar uma solução completa, tecnicamente fundamentada e demonstrável.

A. Integração

- Validar a integração de todos os módulos.
- Executar os cenários completos.
- Validar os requisitos obrigatórios.
- Executar testes de regressão.

- Confirmar que o ambiente é reproduzível.
- Confirmar a ausência de configurações manuais indispensáveis.
- Organizar código e documentação.
- Remover dependências e ficheiros temporários desnecessários.

B. Relatório científico

- Introdução e contextualização.
- Arquitetura da solução.
- Especificação dos protocolos.
- Formatos das mensagens.
- Diagramas de sequência.
- Estratégias de encaminhamento.
- Implementação.
- Metodologia experimental.
- Resultados e gráficos.
- Discussão crítica.
- Conclusões.
- Trabalho futuro.
- Referências bibliográficas.
- Verificação do limite de 12 páginas.

C. Apresentação

- Explicar o problema original.
- Demonstrar a arquitetura.
- Justificar as decisões técnicas.
- Explicar os protocolos.
- Demonstrar a entrada de clientes.
- Demonstrar distribuição de fluxos.
- Demonstrar uma alteração ou falha de rede.
- Apresentar resultados experimentais.
- Explicar limitações e possíveis melhorias.
- Preparar respostas a perguntas técnicas.

Critério de aprovação: os três membros devem ser capazes de explicar e defender a solução completa, incluindo as decisões de arquitetura e os resultados.

PARTE IV — Como vamos trabalhar ao longo do projeto

7. Não vamos executar comandos sem compreender o que fazem

Quero propor uma regra para a nossa metodologia:

> Cada comando, configuração, protocolo e algoritmo deverá ter uma explicação funcional e uma justificação técnica.

Por exemplo, quando chegarmos à configuração de uma interface:

```
ip addr add 10.0.1.2/24 dev eth0
```

Não vamos simplesmente copiar o comando.

Vamos compreender:

Elemento	Significado
`ip`	Ferramenta Linux de configuração de rede
`addr`	Operações relacionadas com endereços
`add`	Adiciona um endereço
`10.0.1.2`	Endereço IPv4 atribuído
`/24`	Comprimento do prefixo de rede
`dev`	Indica que será especificada uma interface
`eth0`	Interface onde o endereço será configurado

Também discutiremos por que utilizamos essa rede, como o prefixo influencia o encaminhamento, o que acontece se configurarmos um endereço incorreto e como confirmar a configuração.

O nosso método de aprendizagem

Para cada funcionalidade, seguiremos esta sequência:

1. Problema: o que precisamos de resolver?

2. Fundamento: qual é o conceito de redes relacionado?

3. Alternativas: que soluções existem?

4. Decisão: qual utilizaremos e porquê?

5. Implementação: como concretizaremos a solução?

6. Validação: como sabemos que funciona?

7. Limitações: em que condições poderá falhar?

8. Documentação: como justificaremos a decisão no relatório?

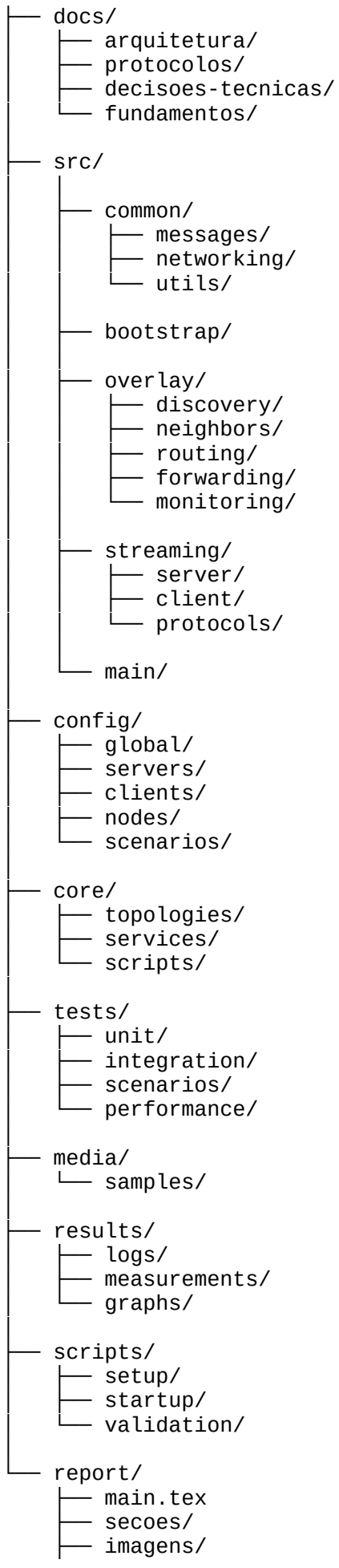
Assim, o código será consequência de uma decisão técnica fundamentada.

---

8. Organização do código e das configurações

Além da estrutura LaTeX que já estabelecemos, sugiro organizarmos o repositório completo da seguinte forma:

```
ESR-TP2/
│
├── README.md
├── .gitignore
│
```


docs/
arquitetura/
protocolos/
decisoes-tecnicas/
fundamentos/
src/
common/
messages/
networking/
utils/
bootstrap/
overlay/
discovery/
neighbors/
routing/
forwarding/
monitoring/
streaming/
server/
client/
protocols/
main/
config/
global/
servers/
clients/
nodes/
scenarios/
core/
topologies/
services/
scripts/
tests/
unit/
integration/
scenarios/
performance/
media/
samples/
results/
logs/
measurements/
graphs/
scripts/
setup/
startup/
validation/
report/
main.tex
secoes/
imagens/

```
├── tabelas/
└── bibliografia/
```

Esta estrutura é uma proposta inicial; os nomes dos módulos e ficheiros definitivos dependerão da linguagem escolhida.

Porque separar dessa forma?

A pasta `src/` contém o código da aplicação.

A pasta `config/` contém parâmetros que poderão mudar sem precisarmos de alterar o código.

A pasta `core/` contém as topologias e os elementos necessários para executar o ambiente de emulação.

A pasta `tests/` contém testes automatizados.

A pasta `results/` armazena as evidências experimentais.

A pasta `report/` contém o artigo científico.

Esta separação permite mudar a topologia, o número de nós e os parâmetros da rede sem reescrever a aplicação.

É particularmente importante para o nosso objetivo de suportar diferentes cenários.

---

9. Como garantir que a solução seja verdadeiramente adaptável?

Esta é uma das decisões arquiteturais mais importantes.

Não devemos desenvolver um programa que dependa dos endereços ou das posições dos nós de uma topologia específica.

Por exemplo, seria inadequado ter lógica semelhante a:

```
if client_id == "C1":    next_hop = "O3"
```

Esse código pressupõe conhecimento de uma topologia específica.

Se mudarmos a localização do cliente, poderemos ter de alterar o programa.

O que pretendemos é uma decisão baseada no estado atual da rede:

```
next_hop = routing_table.get_next_hop(    stream_id)
```

Este segundo exemplo é apenas ilustrativo: a interface definitiva dependerá do modelo de encaminhamento.

Aqui, a aplicação consulta informação de encaminhamento em vez de assumir que o cliente se encontra numa posição predeterminada.

Princípios que devemos respeitar

Princípio	Resultado pretendido
Independência da topologia	Funcionar em grafos diferentes
Identificação lógica dos nós	Não depender de endereços fixos no código
Configuração externa	Alterar parâmetros sem recompilar
Encaminhamento dinâmico	Adaptar os caminhos
Estado por fluxo	Suportar diferentes conteúdos
Separação entre controlo e dados	Reduzir acoplamento
Descoberta automática	Simplificar a entrada de nós
Monitorização	Identificar mudanças
Modularidade	Facilitar testes e evolução
Automatização	Garantir reprodutibilidade

Não significa que a rede funcionará independentemente dos recursos disponíveis. Significa que a implementação não dependerá de uma topologia rígida ou de um número fixo de clientes.

---

10. Organização dos três membros

Em vez de atribuir um terço do projeto isoladamente a cada elemento, recomendo dividir as responsabilidades principais, mantendo conhecimento partilhado.

Área	Responsável principal	Participação dos restantes
CORE e underlay	Membro 1	Configuração e testes
Descoberta e bootstrap	Membro 1	Integração
Protocolo de controlo	Membro 2	Revisão
Encaminhamento e métricas	Membro 2	Testes
Streaming	Membro 3	Integração
Testes de desempenho	Membro 3	Execução e análise
Arquitetura global	Todos	Todos
Relatório	Todos	Todos
Apresentações	Todos	Todos

O mais importante é que nenhum membro fique restrito à sua parte.

Numa apresentação, qualquer um poderá ser questionado sobre o sistema completo.

11. As referências teóricas

Vamos utilizar principalmente Tanenbaum e Kurose/Ross, relacionando cada decisão com os conceitos fundamentais.

Tema	Fundamentação principal
TCP e UDP	Transporte fim a fim
Overlay	Arquiteturas de redes e aplicações distribuídas
Streaming	Aplicações multimédia e requisitos temporais
Routing	Algoritmos de encaminhamento

Tema	Fundamentação principal
Multicast	Distribuição de dados para múltiplos destinatários
QoS	Atraso, jitter, perda e capacidade
Congestionamento	Gestão e utilização dos recursos
Escalabilidade	Distribuição da carga e eficiência
Resiliência	Deteção de falhas e convergência

A correspondência com capítulos e páginas será feita de acordo com as edições efetivamente utilizadas, para não introduzirmos referências bibliográficas incorretas.

Além disso, manteremos um **registo de decisões técnicas**, por exemplo:

Decisão	Justificação	Alternativa considerada	Como validar
Bootstrapper	Facilitar descoberta inicial	Vizinhos manuais	Entrada de nós
Multicast aplicacional	Reduzir tráfego redundante	Unicast direto	Comparação de tráfego
Tabela por fluxo	Suportar múltiplos vídeos	Rota global única	Fluxos simultâneos
Monitorização de ligações	Detetar degradação	Rotas estáticas	Testes de falha

Esse registo servirá de base à escrita do relatório.

PARTE V — O nosso controlo de qualidade

12. Critérios para considerar o projeto concluído

A seguinte tabela será a nossa referência durante todo o desenvolvimento.

Critério	Teste de aceitação
Reprodução do ambiente	Reconstruir os cenários sem reconfiguração manual
Descoberta	Novo nó consegue entrar no overlay
Dinamismo	Nós podem entrar e sair durante a execução
Múltiplos servidores	Dois servidores operacionais
Múltiplos fluxos	Pelo menos dois vídeos distintos por servidor
Múltiplos clientes	Clientes simultâneos com diferentes subscrições
Independência da posição	Clientes funcionam em diferentes pontos da rede
Eficiência	Comparação mensurável com unicast direto
Rotas	Tabelas corretas e sem ciclos de encaminhamento
Métricas	Seleção de rotas baseada em medições
Recuperação	Recuperação perante falhas com alternativas disponíveis
Streaming	Reprodução efetiva de vídeo
Monitorização	Logs e medições disponíveis
Escalabilidade	Resultados com número crescente de clientes
Documentação	Decisões fundamentadas e reproduzíveis

Critério	Teste de aceitação
Defesa	Todos os membros compreendem a solução

Também vamos classificar cada requisito com um estado:

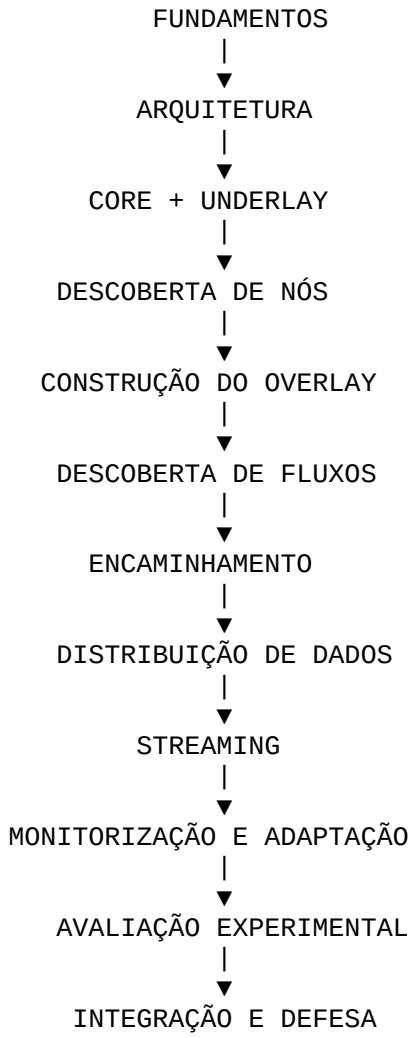
- ⬜ Não iniciado
- 🟡 Em desenvolvimento
- 🔵 Implementado, mas não validado
- 🟢 Implementado e validado
- 🔴 Bloqueado

Só consideraremos um requisito concluído quando estiver validado por um teste.

---

13. Caminho crítico do trabalho

Existem dependências entre os checkpoints. A sequência principal será:



Embora exista esta sequência lógica, algumas atividades poderão decorrer em paralelo.

Por exemplo, podemos implementar um protótipo de streaming simples enquanto desenvolvemos o overlay, integrando os componentes posteriormente.

Isso está alinhado com o próprio enunciado, que esclarece que as etapas sugeridas não precisam de ser executadas obrigatoriamente de forma sequencial.

---

14. O que considero essencial para distinguirmos o trabalho

Gostaria que conseguíssemos demonstrar três níveis de qualidade.

Nível 1 — Correção

O sistema funciona de acordo com a especificação.

Os nós comunicam, as rotas são construídas e os clientes reproduzem os vídeos.

Nível 2 — Eficiência e robustez

O sistema evita replicações desnecessárias, suporta clientes simultâneos e reage a alterações relevantes da rede.

Nível 3 — Qualidade científica

Conseguimos explicar e demonstrar:

- Porque o sistema funciona.
- Porque escolhemos determinados protocolos.
- Porque uma rota é preferível a outra.
- Qual é o custo da manutenção do overlay.
- Qual é o ganho relativamente ao unicast.
- Como o desempenho varia com o número de clientes.
- Que limitações continuam presentes.

Este terceiro nível é o que transforma uma implementação funcional num trabalho sólido de Engenharia de Redes.

---

15. Por onde vamos começar

A nossa próxima atividade deve ser exclusivamente o **Checkpoint 0 — Fundamentos e desenho da arquitetura**.

Antes de abrir o CORE para começar a configurar dispositivos, vamos produzir quatro elementos:

1. Especificação do sistema: identificar precisamente o que cada componente precisa de fazer, quais os requisitos obrigatórios e quais as extensões que pretendemos desenvolver.

2. Diagrama completo da arquitetura: representar os dois servidores, os clientes, os nós overlay, o bootstrapper, as comunicações de controlo e os fluxos multimédia.

3. Especificação inicial dos protocolos: identificar os tipos de mensagens e o ciclo de vida dos nós e dos fluxos, ainda sem programar.

4. Análise das decisões arquiteturais: comparar as alternativas possíveis e justificar a arquitetura escolhida com os fundamentos de redes.

Depois disso, construiremos a primeira topologia no CORE, com persistência desde o início.

A nossa regra principal

Primeiro compreendemos. Depois projetamos. Em seguida implementamos. Finalmente medimos, validamos e justificamos.

É assim que sugiro conduzirmos o projeto: **um checkpoint de cada vez, sem perder a visão do sistema completo e sem avançar enquanto os fundamentos e os critérios de aceitação da etapa atual não estiverem claros.**

CRITÉRIO OBRIGATORIO EM TODO O TRABALHO:

Regra definitiva: bootstrapper apenas para arranque

Esta é uma restrição arquitetural obrigatória do nosso projeto.

O *bootstrapper* terá uma única responsabilidade: **permitir que um nó descubra os seus primeiros vizinhos quando entra na rede overlay.**

Depois desse contacto inicial, a rede deve funcionar de forma distribuída, sem depender do bootstrapper.

Operação	Bootstrapper
Receber o contacto inicial de um nó	✅ Permitido
Fornecer uma lista inicial de vizinhos	✅ Permitido
Calcular rotas	❌ Proibido
Distribuir tabelas de encaminhamento	❌ Proibido
Monitorizar a rede	❌ Proibido
Gerir fluxos multimédia	❌ Proibido
Decidir mudanças de percurso	❌ Proibido
Coordenar a recuperação de falhas	❌ Proibido
Encaminhar pacotes multimédia	❌ Proibido